

Simple Model of Neuron Activation

— Project Report —
Simulation Based Inference

Quang Vinh Nguyen, 231844

Sadat Mahmud, 230749

August 17, 2025

TU Dortmund University

1 Introduction

Neurons are the fundamental signaling units of the nervous system, transmitting information through electrical impulses called action potentials. Since the mid-20th century, following the pioneering work of Hodgkin and Huxley on giant squid axons (Hodgkin & Huxley, 1939), neuro scientists have sought to understand how action potentials arise from the interaction of input currents, ion conductivity, and the electrical properties of the membrane.

A central task in computational and experimental neuroscience is the estimation of intrinsic neuronal parameters such as the membrane time constant and membrane resistance. These quantities determine how a neuron integrates synaptic inputs and shapes its excitability. Accurately recovering them is essential for interpreting electrophysiological recordings, constructing realistic models, and linking cellular properties to network dynamics. To study this inference problem in a controlled setting, we use the Leaky Integrate-and-Fire (LIF) model - a simplified but biologically motivated representation of neuronal membrane dynamics. The LIF model combines continuous subthreshold integration and leakage with a discrete threshold-triggered spike and reset rule. Its simplicity makes it computationally efficient while preserving essential features of neuronal excitability.

In this project, based on the tutorial from Neuromatch Academy we simulate voltage traces from the LIF model under a fixed step current input and apply BayesFlow (BayesFlow Team), a simulation-based Bayesian inference framework using neural network, to learn the mapping from observed data to posterior distributions over the membrane time constant and membrane resistance. The trained neural posterior estimator is then evaluated on new, unseen simulation data, which is also simulated, to assess its ability to recover the original parameters. The results show that BayesFlow can recover the parameters relatively well.

2 Data & Statistical model

Before describing how the data is generated, we want to explain the Leaky Integrate-and-Fire (LIF) model and its parameters. The explanation focuses on the relationship between the parameters represented by mathematical equations. The biological meaning behind it can be referred to Dayan and Abbott (2001). The LIF model describes the subthreshold membrane potential $V(t)$ of a neuron as a first-order differential equation:

$$\tau_m \frac{dV}{dt} = -(V(t) - V_{\text{rest}}) + R \cdot I(t), \quad (1)$$

where τ_m is the membrane time constant, R the membrane resistance, V_{rest} the resting potential, and $I(t)$ the input current. The equation reflects two opposing processes: passive leakage of the membrane voltage back toward rest ($V(t) - V_{\text{rest}}$), and integration of synaptic or external current input scaled by the resistance ($R \cdot I(t)$). When the voltage reaches a threshold V_{th} , the model emits a spike - a action potentials is generated - and the membrane potential is reset to V_{reset} .

In this study, the membrane time constant τ_m and the resistance R serve as the target variables for inference and should be generated first. They are not directly sampled but are instead derived from more basic physiological quantities to preserve physiological realism. The specific membrane capacitance c_m is highly conserved across biological membranes, with a typical value of about 10 nF/mm^2 (Dayan & Abbott, 2001, p.156). To reflect this property, it is modeled as $c_m \sim \text{lognormal}(\ln(10), 0.05^2)$, where the narrow standard deviation ensures only minor deviations from the physiological mean. In contrast, the specific membrane resistance r_m varies more substantially across neurons, typically around $1 \text{ M}\Omega \cdot \text{mm}^2$ (Dayan & Abbott, 2001, p.158). This variability is represented by $r_m \sim \text{lognormal}(\ln(1), 0.5^2)$, which allows for a wider spread of plausible values. The membrane surface area is fixed at $A = 0.05 \text{ mm}^2$, corresponding to a moderate cell size within the biological range of $0.01\text{-}0.1 \text{ mm}^2$. With this construction, the derived parameters $\tau_m = r_m \cdot c_m$ and $R = r_m/A$ fall within realistic physiological ranges, while maintaining the natural coupling dictated by biophysics.

Next, to simplify the data generation process, other parameters in Eq. 1 are kept fixed at biologically plausible values. The resting potential and the reset potential are both set to $V_{\text{rest}} = V_{\text{reset}} = -65.0 \text{ mV}$, while the firing threshold is fixed at $V_{\text{th}} = -50.0 \text{ mV}$ (see Dayan and Abbott's example, p.164). In practice, however, the reset potential V_{reset} and the resting potential V_{rest} are not necessarily identical; depending on the biophysical context, a neuron may reset to a potential distinct from its resting level. Furthermore, the input currents are also simplified. Each simulation consists of $T = 1000$ discrete time steps of size Δt . These input currents follow rectangular pulses: they stay at a baseline of 0 nA at the start, rise to a constant 1 nA between steps $t = 300$ and $t = 700$, and then return to baseline. To mimic biological variability and measurement noise, Gaussian white noise with standard deviation of 0.05 nA is added, introducing stochastic fluctuations into the membrane potential trajectories.

Finally, training and validation data are generated by simulating voltage trajectories from the LIF model (Eq. 1) under a controlled stimulation protocol. The membrane dynamics are approximated using a forward-Euler integration scheme (referred to the Neuromatch Academy's tutorial), where the potential $V(t)$ is updated according to

$$V(t + \Delta t) = V(t) + \frac{\Delta t}{\tau_m} [-(V(t) - V_{\text{rest}}) + R \cdot I(t)] .$$

A spike is registered whenever the updated membrane potential exceeded the firing threshold, $V(t + \Delta t) \geq V_{\text{th}}$. In such cases, the voltage is reset instantaneously to $V_{\text{reset}} = V_{\text{rest}}$, and the membrane potential is recharged again from this potential level. Many LIF formulations include a refractory period, during which the neuron is prevented from firing again immediately after a spike. This is often implemented by clamping the voltage at V_{reset} for several time steps following threshold crossing. While such a mechanism increases biological realism by preventing unrealistically high firing rates, it is not strictly required in the basic LIF model.

Figure 1 illustrates the spiking regime of the leaky integrate-and-fire model. During the stimulus period the membrane potential rises and repeatedly crosses the threshold at -50 mV. Each crossing triggers the emission of a spike, after which the voltage is reset to the resting potential of -65 mV. This sequence produces a characteristic train of action potentials. In contrast, Figure 2 shows the subthreshold regime. Here the depolarization remains below the threshold throughout the stimulus. No spikes are generated, and the voltage instead approaches a plateau before relaxing back to rest once the stimulus ends.

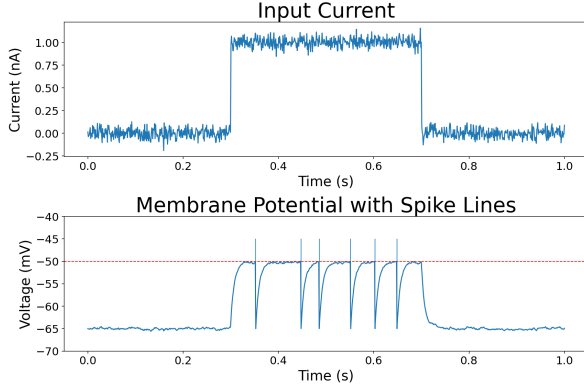


Figure 1: Membrane potential with spiking.

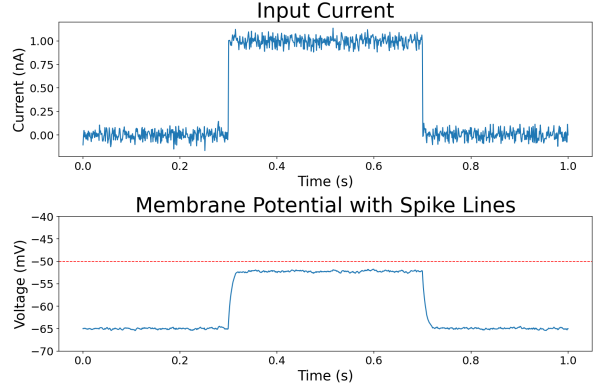


Figure 2: Membrane potential without spiking.

3 Approximator & Training

The approximator is composed of three main components: an adapter that organizes the simulator outputs into the required input format, a summary network that extracts informative features from the voltage trajectories and an inference network that maps these features to a posterior distribution over the parameters. Several small adjustments in each component were tested to find the variant with the best performance and will also be discussed in this section to reason about the choice of the final variant.

In this project, τ_m and R serve as the inference variables, while the membrane potential trace acts as the conditional variable. The two inference parameters are fed directly into the model without transformation or standardization. Although preliminary experiments included logarithmic transforms and standardization, these modifications did not improve performance and were therefore omitted in the final approximator.

The membrane potential trace is processed by the summary network. Although recurrent architectures such as GRUs or LSTMs are well suited for time-series data, they are computationally expensive. Instead, the summary network is implemented as a multilayer perceptron with four hidden layers of decreasing size (400, 200, 100, and 50 units), each followed by a ReLU activation function. No additional regularization techniques, such as dropout, batch normalization, or weight decay, are applied. Given the simplified structure of the input current pulses, this feed-forward architecture is sufficient to capture the relevant features of the data.

The inference network is a flow-based model composed of a sequence of invertible transformations, including coupling layers and learned permutations. Within each coupling layer, the transformations are parameterized by a subnet. The model transforms a simple base distribution, usually a standard normal, into a flexible posterior distribution capable of capturing complex shapes. In this project, the default configuration of BayesFlow’s CouplingFlow is used, which consists of an MLP subnet with default widths, six invertible layers, affine transformations in the coupling layers, random permutations between layers, ActNorm applied before each coupling, and a standard normal base distribution (see the documentation of BayesFlow Team).

The approximator is trained offline on data generated by the simulator. A total of 1000 simulations are used for training and an additional 1000 simulations are held out for validation. Model performance is monitored on the validation set throughout training to prevent overfitting and to ensure that the learned posterior generalizes to unseen simulations. The optimizer is Adam with a learning rate of 0.0005 by default (see the documentation of BayesFlow Team). No additional regularization methods are applied. Training is performed for 500 epochs, using a batch size of 64.

4 Diagnostics

Convergence

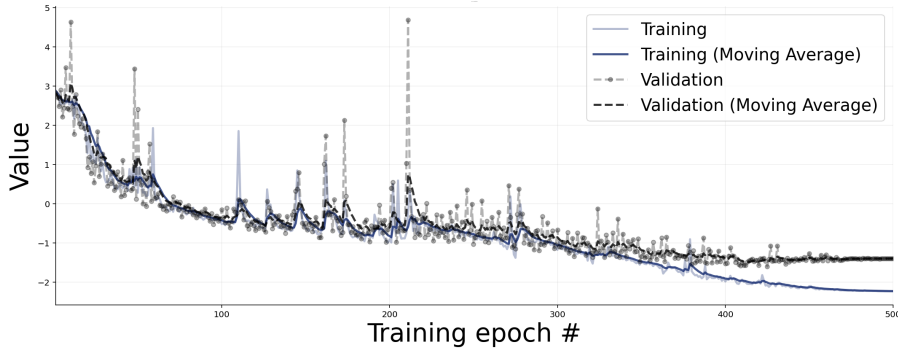


Figure 3: Loss trajectories for training and validation over 500 epochs

Figure 3 shows both trajectories with smoothed averages. A steady downward trend is observed for the training loss, while the validation loss initially follows but then flattens out earlier. This indicates that the model continues to fit the training data more closely over time, but the ability to generalize to unseen simulations reaches a plateau. Occasional short spikes are visible in the raw curves, which can be attributed to stochastic mini-batching and simulation noise; these are smoothed out in the moving averages and do not change the overall pattern. The divergence between decreasing training loss and flat validation loss suggests that the model capacity is sufficient, but additional training does not yield further improvements in performance.

Simulation-based Calibration

Simulation-based calibration (SBC) provides a diagnostic to assess whether the amortized posterior approximator is well-calibrated. The method relies on repeatedly simulating parameters from the prior, generating data from the simulator, inferring the posterior distribution, and recording the rank of the true parameter within posterior samples (Talts et al., 2020). If the posterior is correctly specified and learned, these rank statistics should follow a uniform distribution.

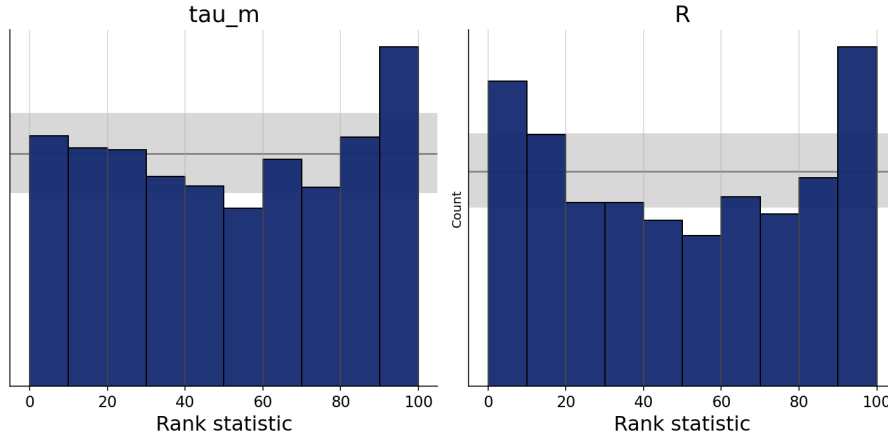


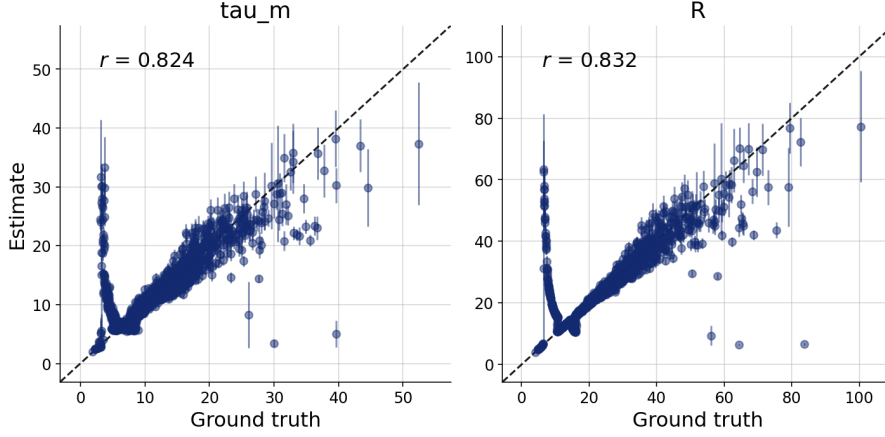
Figure 4: SBC Histograms of τ_m and R

In our results (Figure 4), both τ_m and R show slightly U-shaped histograms, suggesting a mild tendency toward underdispersion. Slight deviations from flatness are visible at the extremes, where edge ranks are slightly overrepresented. This suggests that while the approximator captures the main posterior structure, there may be biases in regions corresponding to very small or very large parameter values.

Parameter Recovery

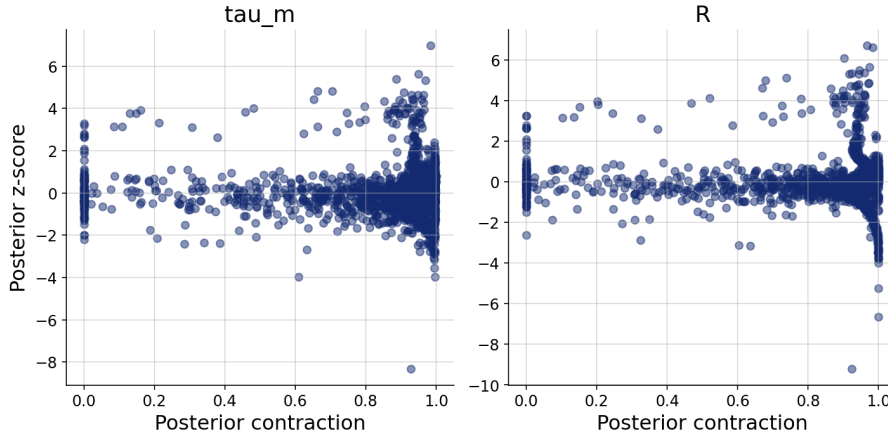
Parameter recovery assesses how well the approximator can reproduce the true values of τ_m and R from simulated datasets. Figure 5 compares the posterior means against the ground-truth parameters, with error bars indicating posterior standard deviations. Both parameters show strong positive correlations with the true values ($r = 0.824$ for τ_m and $r = 0.832$ for R), demonstrating that the model extracts meaningful information from the voltage traces.

However, deviations from the diagonal line reveal some systematic biases. At small ground-truth values, both τ_m and R are overestimated: points lie well above the diagonal and appear to be arbitrarily estimated. In the mid-range the estimates track the diagonal closely, while at the upper end there is a mild downward bias (underestimation) and increased spread.

Figure 5: Parameter recovery Diagnostics of τ_m and R

Posterior z-score and posterior contraction

The Z-score vs. contraction diagnostic (Schad et al., 2020) provides a joint view of bias and informativeness in posterior estimates. The posterior Z-score is defined as $(\text{posterior mean} - \text{true parameter}) / \text{posterior std}$, which measures how well the estimated posterior mean matches the true parameter used for simulating the data. Ideally, Z-scores are centered around zero, indicating unbiased estimation. Posterior contraction is defined as $1 - (\text{posterior variance} / \text{prior variance})$, which quantifies the reduction in variance when moving from the prior to the posterior. A contraction value close to one signals strong information gain, while values near zero imply that the posterior variance remains similar to the prior.

Figure 6: Posterior Z-score vs. Contraction Diagnostics of τ_m and R

In Figure 6, both τ_m and R show Z-scores broadly concentrated around zero, suggesting no major systematic overall bias. Contraction values cluster near one, indicating that the data provides substantial information about both parameters. However, the spread

of Z-scores increases at high contraction, with some instances showing overly confident posteriors that deviate from the ground truth. This points to a tendency toward slight underdispersion, consistent with earlier SBC results, but the overall pattern supports effective learning and informative posteriors.

5 Discussion & Conclusion

This work investigated whether amortized Bayesian inference with BayesFlow can recover the key parameters of a leaky integrate-and-fire (LIF) neuron model from simulated voltage trajectories. The framework combined a feed-forward summary network with a conditional normalizing flow to approximate posterior distributions over the membrane time constant τ_m and the membrane resistance R .

Overall, the results indicate that the approximator captures much of the parameter dependence encoded in the data. Diagnostics such as simulation-based calibration, parameter recovery, and Z-score vs. contraction plots show broadly accurate inference and informative posteriors. At the same time, systematic deviations remain. In particular, recovery is less reliable at the extremes of the parameter space:

The strange distribution of data points in the initial parameter recovery may come from the data used. Small values of the membrane time constant τ_m cause the membrane potential to increase very quickly. The simulated data of the membrane potential trace then becomes just a series of constants because the potential is reset repeatedly. On the contrary, small values of the membrane resistance R produce only minimal voltage deflections that are easily blurred by noise.

From a methodological perspective, the choice of summary network plays an important role. The use of a simple multilayer perceptron demonstrated that even a generic architecture can learn useful embeddings of voltage trajectories under simplified input currents. However, time series-aware models such as GRUs, LSTMs, or convolutional architectures, although computationally expensive, can better exploit sequential dependencies and improve parameter resilience if trained long enough. Similarly, inference networks, although flexible in the default CouplingFlow configuration, can be improved with more expressive transformations such as spline concatenation, which can potentially mitigate underdispersion.

In conclusion, BayesFlow provides a promising tool for simulation-based inference in neuroscience models. At the same time, this study also highlights the challenges in fine-tuning both simulation design and neural architecture to improve the performance of approximators. Extending amortized simulation-based inference to these richer settings could provide a powerful tool for data-driven parameter estimation in computational neuroscience, but will require careful balancing of model complexity, simulation cost, and network expressiveness.

References

- Hodgkin, A. L., & Huxley, A. F. (1939). Action potentials recorded from inside a nerve fibre. *Nature*, 144(3651), 710–711. <https://doi.org/10.1038/144710a0>
- Dayan, P., & Abbott, L. F. (2001). *Theoretical neuroscience: Computational and mathematical modeling of neural systems*. MIT Press.
- Schad, D. J., Betancourt, M., & Vasisht, S. (2020). Toward a principled bayesian workflow in cognitive science. *Psychological Methods*, 26(1), 103–126. <https://arxiv.org/abs/1904.12765>
- Talts, S., Betancourt, M., Simpson, D., Vehtari, A., & Gelman, A. (2020). Validating bayesian inference algorithms with simulation-based calibration. <https://arxiv.org/abs/1804.06788>
- BayesFlow Team. (n.d.). *Bayesflow: A framework for amortized bayesian inference*. <https://bayesflow.org/>
- Neuromatch Academy. (n.d.). *W2d3: Biological neuron models — tutorial 1* [Accessed: 2025-08-01]. https://compneuro.neuromatch.io/tutorials/W2D3_BiologicalNeuronModels/student/W2D3_Tutorial1.html